



Plan de Acción Definitivo: Arquitectura de Base de Datos en Supabase

Distribución de Tareas (2 Personas)

- **Persona A (Diseño de Tablas y Relaciones):** Ejecutará el código SQL para crear las tablas y las relaciones. Llenará a mano algunos productos de prueba en el catálogo para que el Frontend pueda empezar a probar.
- **Persona B (Seguridad RLS y Configuración API):** Activará las políticas de seguridad (RLS) para que nadie pueda borrar la base de datos desde el navegador, y entregará las llaves de acceso al equipo Frontend.

Diseño del Esquema (Diccionario de Datos)

Este es el diseño relacional de la base de datos. Consta de 3 tablas principales (2 de dimensiones y 1 de hechos para la telemetría).

1. Tabla: usuarios_sesion (Dimensión de Usuario)

Almacena el perfil demográfico anónimo de cada estudiante que ingresa a la tienda.

- `id (UUID)`: Llave primaria generada automáticamente. Identificador único de la sesión del alumno.
- `carrera (VARCHAR)`: Carrera del estudiante (Ej. "Ingeniería de Datos", "Sistemas Embebidos").
- `genero (VARCHAR)`: Género del estudiante (Ej. "Masculino", "Femenino").
- `created_at (TIMESTAMP)`: Fecha y hora exacta de su registro.

2. Tabla: catalogo_productos (Dimensión de Producto)

Contiene la información estática de los artículos que se venden. Debe ser pre-llenada por el equipo.

- `id (VARCHAR)`: Llave primaria. Un ID legible que también se usará en la URL (Ej. "macbook-pro-m3").
- `nombre (VARCHAR)`: Nombre completo del producto.
- `categoria_general (VARCHAR)`: Clasificación principal ("Académico" o "Entretenimiento").
- `carrera_objetivo (VARCHAR)`: Carrera a la que va dirigida el producto (Ej. "Todas", "Sistemas Embebidos").
- `tiene_gpu_dedicada (BOOLEAN)`: `true` si es un equipo con gráficos dedicados (RTX), `false` si no lo es.
- `precio_original (DECIMAL)`: Precio base del producto.

- `porcentaje_descuento` (*DECIMAL*): Descuento aplicable en formato decimal (Ej. 0.15 = 15%).
- `created_at` (*TIMESTAMP*): Fecha en la que se dio de alta el producto.

3. Tabla: `interacciones_telemetria` (Tabla de Hechos)

El corazón del proyecto. Registra cada acción (clic, tiempo, compra) que los usuarios hacen con los productos.

- `id` (*UUID*): Llave primaria del evento.
- `usuario_id` (*UUID*): Llave foránea (Foreign Key) vinculada a `usuarios_sesion.id`.
- `producto_id` (*VARCHAR*): Llave foránea vinculada a `catalogo_productos.id`.
- `tipo_evento` (*VARCHAR*): Acción específica. Puede ser: `'view'` (ver), `'add_to_cart'` (añadir al carrito) o `'purchase'` (comprar).
- `dwell_time_segundos` (*DECIMAL, Nullable*): Segundos que pasó viendo el producto. Se llena solo si el evento es `'view'` o antes de un `'add_to_cart'`.
- `precio_pagado` (*DECIMAL, Nullable*): Precio final. Se llena solo si el evento es `'purchase'`.
- `metodo_pago` (*VARCHAR, Nullable*): Cómo pagó. Se llena solo si el evento es `'purchase'` (Ej. "Tarjeta", "Beca Benito Juárez").
- `es_fin_de_semana` (*BOOLEAN*): `true` si la acción ocurrió en sábado o domingo.
- `created_at` (*TIMESTAMP*): Marca de tiempo exacta del evento.

Código SQL para Crear las Tablas (Para la Persona A)

Vayan al *SQL Editor* en Supabase, abran un "New Query" y peguen este código. Ejecútenlo todo junto.

```
-- 1. Crear tabla de Perfiles de Estudiantes (Usuarios)
CREATE TABLE public.usuarios_sesion (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  carrera VARCHAR(100) NOT NULL, -- Ej: 'Ingeniería de Datos'
  genero VARCHAR(50) NOT NULL, -- Ej: 'Masculino', 'Femenino'
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Crear tabla de Catálogo de Productos
CREATE TABLE public.catalogo_productos (
  id VARCHAR(100) PRIMARY KEY, -- Identificador amigable (ej: 'laptop-rtx-407'
  nombre VARCHAR(255) NOT NULL,
  categoria_general VARCHAR(50) NOT NULL, -- 'Académico' o 'Entretenimiento'
  carrera_objetivo VARCHAR(100) DEFAULT 'Todas', -- Ej: 'Sistemas Embebidos',
  tiene_gpu_dedicada BOOLEAN DEFAULT false,
  precio_original DECIMAL(10, 2) NOT NULL,
  porcentaje_descuento DECIMAL(3, 2) DEFAULT 0.00, -- 0.15 significa 15% de d
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Crear tabla de Telemetría (Eventos)
CREATE TABLE public.interacciones_telemetria (
```

```

id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
usuario_id UUID REFERENCES public.usuarios_sesion(id) ON DELETE CASCADE,
producto_id VARCHAR(100) REFERENCES public.catalogo_productos(id) ON DELETE

-- El evento puede ser: 'view', 'add_to_cart', o 'purchase'
tipo_evento VARCHAR(50) NOT NULL,

-- Columnas dinámicas (se llenan dependiendo del tipo de evento)
dwell_time_segundos DECIMAL(10, 2), -- Se llena antes de un 'add_to_cart' o
precio_pagado DECIMAL(10, 2),        -- Solo se llena en evento 'purchase'
metodo_pago VARCHAR(100),            -- Solo en 'purchase' (Ej: 'Tarjeta', '
es_fin_de_semana BOOLEAN,            -- Puede mandarlo el frontend o calcula

created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Índices para optimizar velocidad de consultas analíticas
CREATE INDEX idx_telemetria_usuario ON public.interacciones_telemetria(usuario_
CREATE INDEX idx_telemetria_producto ON public.interacciones_telemetria(product
CREATE INDEX idx_telemetria_evento ON public.interacciones_telemetria(tipo_even

```

Configuración de Seguridad RLS (Para la Persona B)

Abran otro "New Query" y ejecuten este código para blindar la base de datos contra hackers o alumnos curiosos:

```

-- Activar RLS en las tablas expuestas al frontend
ALTER TABLE public.usuarios_sesion ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.interacciones_telemetria ENABLE ROW LEVEL SECURITY;

-- Permitir que CUALQUIERA (anon) pueda INSERTAR un nuevo perfil al entrar a la
CREATE POLICY "Permitir registro de usuarios anonimos"
ON public.usuarios_sesion FOR INSERT TO anon WITH CHECK (true);

-- Permitir que CUALQUIERA (anon) pueda INSERTAR un evento de telemetría o comp
CREATE POLICY "Permitir tracking anonimo"
ON public.interacciones_telemetria FOR INSERT TO anon WITH CHECK (true);

-- Nota Importante: Como NO hay políticas para SELECT, UPDATE o DELETE,
-- los visitantes de la página web están completamente ciegos. Solo pueden "esc

```

Matriz de Validación: ¿Cómo resolvemos las 8 preguntas?

Esta tabla demuestra al equipo que la base de datos está lista para todos:

1. **Rama (Días de compra):** Se cruza `tipo_evento = 'purchase'` con la columna `es_fin_de_semana`.
2. **Russel (GPUs y Carrera):** Se cruza `dwell_time_segundos` con `carrera` (de usuarios) y `tiene_gpu_dedicada` (de productos).

3. **Edgardo (Gasto Académico vs Ocio):** Se suma `precio_pagado` (cuando `tipo_evento='purchase'`) agrupado por `categoria_general` (Académico/Entretenimiento).
4. **Damian Novelo (Descuentos >= 15%):** Se calcula la tasa de conversión (`purchase / view`) agrupando por `porcentaje_descuento >= 0.15` .
5. **Lukaku Jr (Dwell time previo al carrito):** Se mide el `dwell_time_segundos` enviado en el momento exacto en que `tipo_evento = 'add_to_cart'` , agrupado por `categoria_general` .
6. **Bianca Acosta (Compras por Género):** Se cuenta el total de `tipo_evento = 'purchase'` agrupado por `genero` de la tabla usuarios.
7. **Jonathan (Impacto general de descuentos):** Se compara el volumen de `purchase` en productos donde `porcentaje_descuento > 0` vs `porcentaje_descuento = 0` .
8. **Isaac (Beca Benito Juárez y carrera):** Se filtran las compras con `metodo_pago = 'Beca Benito Juárez'` y se revisa si la `carrera` del usuario coincide con la `carrera_objetivo` del producto.